DDPG-Based Two-Timescale Joint Resource Allocation and Channel-Aware Task Offloading for Vehicular Edge-Computing Networks

Yang Xia, *Member, IEEE*, Lijuan Li, *Graduate Student Member, IEEE*, Tong Xue, *Member, IEEE*, Daojun Liang, and Qiyue Li, *Senior Member, IEEE*

***Abstract*—Vehicular edge-computing (VEC) server provides vehicles with sufficient computational resources to meet the low-latency requirements of their tasks. In this article, we study a novel partial offloading scheme in a VEC network, where the task can be unevenly partitioned into multiple subtasks. Different from existing work, we account for the nonarbitrary divisibility of tasks and enable parallel execution of task uploading and computing during offloading. Taking into account task generation time and small-scale channel gain variations at different time scales, a two-timescale dynamic offloading problem is studied with the objective of minimizing the average processing delay of all tasks by jointly optimizing resource allocation and the number of subtasks offloaded. The problem turns out to be a min-max optimization problem. To solve it, a deep deterministic policy gradient (DDPG)-based two-timescale task offloading and resource allocation (DDPG-TTORA) algorithm is proposed to obtain resource allocation decisions at each long time slot and subtask-offloading decisions at each short time slot. Simulation results demonstrated that the proposed DDPG-TTORA algorithm can reduce the average processing delay by 36.75% compared to the full partial offloading (FPO) algorithm.**

***Index Terms*—Deep deterministic policy gradient (DDPG), resource allocation, task offloading, two-timescale, vehicular edge computing (VEC).**

NOMENCLATURE

$T^{link}_{n,i}$	Link connection time between the nth vehicle and the RSU in the ith long time slot.
$t^{u}_{n,m,i,j}$	Uploading delay of the mth subtask in the jth short time slot from the nth vehicle to the RSU.
$t^{c}_{n,m,i,j}$	Computing delay for the mth subtask in the VEC server in the jth short time slot.
$t^{of}_{n,i,j}$	Task-processing delay for completing all subtasks in the jth short time slot.
$t^{idle}_{n,i,j}$	Idle time in the jth short time slot.
$t^{Q}_{n,i,j-1}$	Computational delay for processing the remaining subtasks of the $j-1$th short time slot.
$t^{of}_{n,i}$	Processing delay at the VEC server in the jth long time slot.
$t^{lo}_{n,i}$	Computing delay for processing subtasks locally at the nth vehicle in the ith long time slot.
$t_{n,i}$	Task-processing delay in the ith long time slot.
$D_{n,i}$	Task size of nth task at the ith long time slot.
$D_{n,m,i}$	Task size of the mth subtask of the nth task at the ith long time slot.
$M_{n,i}$	Maximum number of subtasks into which the nth task is divided at the ith long time slot.
$M_{n,i,j}$	Maximum number of subtasks uploaded to the VEC server at jth short time slot.

Received 12 February 2026; revised 2 April 2026; accepted 15 April 2026. Date of publication 20 April 2026; date of current version 29 June 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62572167, Grant 62427809, Grant 52277087, and Grant 62501365; and in part by the Natural Science Foundation of Shandong Province, China, under Grant ZR2025QC1637. *(Corresponding author: Qiyue Li.)*

Yang Xia, Lijuan Li, and Qiyue Li are with the School of Electrical Engineering and Automation, Hefei University of Technology, Hefei, Anhui 230009, China, and also with the Engineering Technology Research Center of Industrial Automation of Anhui Province, Hefei, Anhui 230009, China (e-mail: yangxia@hfut.edu.cn; lilijuan@mail.hfut.edu.cn; liqiyue@mail.ustc.edu.cn).

Tong Xue is with the College of Electronic and Information Engineering, Shandong University of Science and Technology, Qingdao, Shandong 266590, China (e-mail: tongxue@sdust.edu.cn).

Daojun Liang is with the Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), Jinan, Shandong 250103, China (e-mail: liangdj@sdas.org).

Digital Object Identifier 10.1109/JIOT.2026.3685527

I. INTRODUCTION

DRIVEN by the rapid development of vehicular networks and wireless communication technologies, various computation-intensive and latency-sensitive applications have emerged [1], [2], [3], [4]. These applications require substantial computational resources for processing, while the limited resources of vehicles make it difficult to meet their low-latency requirements [5], [6]. To address these demands, edge computing has been introduced as a key solution in the field of vehicular networks [7], [8], [9]. By deploying vehicular edge-computing (VEC) servers in roadside units (RSUs), vehicles can offload their tasks entirely or partially to the RSUs for processing to reduce task-processing delay [10], [11], [12].

Although VEC servers provide vehicles with abundant computing resources, the vehicles first offload their tasks to the VEC servers. Consequently, the task-processing delay includes both the uploading delay for transmitting the task from the vehicle to the RSU and the computing delay for processing

2327-4662 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:06:43 UTC from IEEE Xplore. Restrictions apply.

the task at the VEC server [13], [14]. Therefore, it is crucial to design appropriate task-offloading decisions to reduce task-processing delay.

A. Related Work and Motivation

Numerous studies have been done to design proper offloading decisions for the VEC networks [15], [16], [17], [18], [19]. In [15] and [16], the impact of rapidly time-varying channels caused by vehicle mobility on task-offloading decisions has been investigated. Specifically, Hu et al. [15] employed a path prediction model to estimate the noncausal channel state information, and proposed a receding horizon-based task-offloading method to reduce task-processing delay. Wang et al. [16] utilized a reinforcement learning-based algorithm to cope with the problems of time-varying networks, and designed a double-deep Q-network-based task-offloading algorithm to reduce task-processing delay. Some studies have also examined the issue of limited link connection time between vehicles and VEC servers due to the high mobility of vehicles [17], [18], [19]. For example, Zhao et al. [17] established a mobility model to precisely evaluate the connectivity between vehicles and VEC servers, and proposed a multihop communication task-offloading strategy by selecting the appropriate VEC server to reduce task-processing delay. The limited link connection time can also lead to frequent handovers, causing task-offloading failures. To address this, a task migration scheme was designed, and a deep reinforcement learning (DRL)-based task-offloading algorithm was proposed to reduce task-processing delay [18]. Similarly, Li et al. [19] tackled the issue of task-offloading failure by fully utilizing the computing resources of VEC servers, and developed a mobility-aware task-offloading, communication, and computing resource allocation algorithm to reduce the task-processing delay.

The above works all focus on scenarios where tasks must be executed either entirely locally or entirely on the VEC servers, overlooking the possibility of arbitrarily partitioning a task. A portion of the task is offloaded to VEC servers for processing, i.e., partial offloading. The partial offloading scheme can fully exploit the computing resources of both the VEC server and the vehicle, thereby effectively reducing task-processing delay. However, the resources of the VEC servers are limited and may be insufficient to meet the low-latency requirements of all vehicular tasks when all tasks are offloaded to VEC servers for processing. Therefore, it is necessary to make partial offloading and resource allocation decisions by jointly optimizing task-offloading ratios and resource allocation while meeting the low-latency requirements of all tasks. Several studies have investigated partial offloading and resource allocation decisions in multivehicle VEC networks [20], [21], [22], [23], [24], [25]. For instance, in [20], a partial offloading strategy was proposed by jointly optimizing the offloading ratio, computation, and communication resources to reduce the task-processing delay. Similarly, a vehicle-edge collaborative computing framework was designed by optimizing partial offloading along with computation and communication resource allocation to reduce the task-processing delay [21]. Given the limited computing resources of VEC servers, offloading excessive tasks for processing can easily lead to network congestion. Abbas et al. [22] proposed a decentralized learning-based distributed partial offloading scheme among multiple RSUs by optimizing the offloading ratio, wireless channel allocation, and computing resource allocation to reduce the task-processing delay. Similarly, Lin et al. [23] proposed a scheme of nonorthogonal multiple access (NOMA)-enabled multiedge servers with partial offloading to tackle the overload issue of VEC servers, which allows task-offloading to edge servers (adjacent vehicles and multiple VEC servers) to reduce the task-processing delay. Taking the selfish nature of VEC servers into account, vehicles need to purchase their computing resources to obtain computational services from them. An incentive-driven partial offloading framework was designed in [24], and a DRL-based pricing-driven dynamic resource allocation algorithm was proposed in [25], both of which incentivize edge servers to participate in task offloading, thereby effectively reducing task-processing delay.

Although the above studies have investigated partial offloading strategies, several issues remain unresolved. First, existing studies often assume that the tasks can be arbitrarily divided, while overlooking the fact that some tasks are not arbitrarily divisible. For example, tasks such as image or video processing must be segmented according to their inherent characteristics [26], which significantly influences task-offloading decisions. Second, the prevailing approach in most studies assumes a sequential "uploading-first, computing-later" pattern during task offloading. This results in the computing resources of the VEC server remaining idle during the task uploading, which not only leads to low resource utilization but also has minimal impact on reducing task-processing delay. Third, existing studies typically assume that the channel state information remains quasi-static during the task-uploading process, which oversimplifies the offloading scenario. The millisecond-level variations of small-scale fading channels should be taken into account, with real-time acquisition of channel state information to enable the most accurate task-offloading decisions based on channel conditions. Fourth, since vehicles' arrival at the RSU-covered road segment follows a Poisson process, and the high mobility of vehicles results in limited sojourn time within the cell, the number of vehicles within the cell varies across different time slots. This variation in vehicle numbers determines the changes in task volume, thereby affecting task-offloading and resource allocation decisions. Moreover, the dynamic environment of vehicular networks, characterized by time-varying factors such as the number of vehicles and channel gains between vehicles and VEC servers, poses significant challenges for task-offloading and resource allocation.

B. Contributions and Organization

To this end, a novel two-timescale partial offloading mechanism is proposed, supporting parallel processing of task uploading and computing to fully utilize the computation resources of the VEC server. It is assumed that each vehicle has a divisible task at each time slot. Each task is unevenly segmented into multiple subtasks based on its attributes.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:06:43 UTC from IEEE Xplore. Restrictions apply.

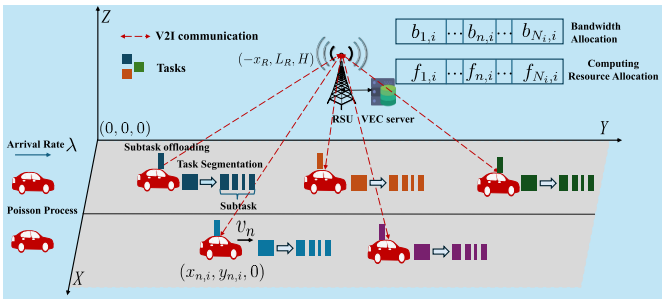


Fig. 1. Process of subtask-offloading and resource allocation decision for a multivehicle VEC network.

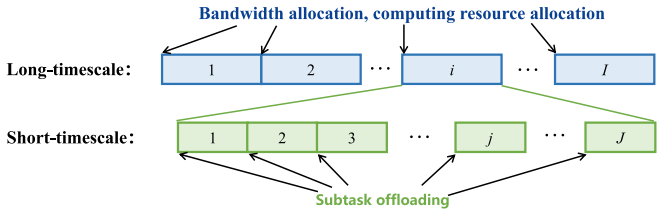


Fig. 2. Illustration of two-timescale joint resource allocation and subtask offloading: 1) bandwidth allocation and computing resource allocation decisions at each long time slot and 2) subtask-offloading decision at each short time slot.

Considering small-scale fading and task generation time at different time scales, the RSU makes subtask-offloading decisions for each vehicle at short time scales and resource allocation decisions at long time scales. The main contributions of this article are summarized as follows.

1) A novel two-timescale task-offloading and resource allocation (TTORA) scheme is proposed for multivehicle VEC networks to minimize the average processing delay of all tasks, where each task is unevenly segmented into multiple subtasks. Unlike the existing work, the task is first not arbitrarily divisible; the divided subtasks are then processed with parallel uploading and computation; moreover, it takes into account that task generation time and small-scale fading variations occur on different timescales.
2) The formulated problem is a min-max optimization problem. To solve it, the problem is transformed into a novel Markov decision process (MDP), where the action space includes the bandwidth and computing resource allocation decisions within each long time slot and the number of subtask offloading within each short time slot. A deep deterministic policy gradient (DDPG)-based TTORA algorithm is proposed to minimize the average processing delay of all tasks.
3) Simulation results show that our proposed DDPG-TTORA algorithm can effectively reduce the average processing delay. The delay is reduced by around 36.75% compared to the FPO algorithm.

The remainder of this article is organized as follows. Section II describes the system model for multivehicle VEC networks. In Section III, a min-max optimization problem is established to achieve the average processing delay minimization. In Section IV, a DDPG-based TTORA algorithm is proposed. Section V presents simulation results and analysis. Finally, we conclude this article in Section VI.

II. System Model

A. System Architecture

In this work, we investigate a multivehicle VEC network with one RSU connected with a VEC server and multiple vehicles within the communication range of the RSU, as shown in Fig. 1. These vehicles are randomly distributed on a unidirectional two-lane road. The arrival of vehicles at the road section covered by the RSU follows a Poisson process with rates λ [27]. It is assumed that the time-slot length for the vehicle generating a task is at the second level (i.e., the time slot is a long time slot), while the time-slot length for block-fading Rayleigh remains constant within a time slot at the millisecond level (i.e., the time slot is a short time slot) [28], [29]. We adopt a discrete time model which is equally divided into I long time slots, where the length of each long time slot is ΔT [30], [31]. The set of vehicles is denoted by $\mathcal{N}_i = \{1, \ldots, N_i\}$ at the ith long time slot. Each vehicle has a delay-sensitive or computation-intensive task at the ith long time slot, which is described as $(D_{n,i}, \delta_{n,i}, \tau_{n,i}^{max})$, where $D_{n,i}$, $c_{n,i}$, and $\tau_{n,i}^{max}$ denote the task size, the task computation density, and the maximum tolerable delay, respectively. It is assumed that each task is divided unevenly into multiple subtasks. The nth task is divided into $M_{n,i}$ subtasks at the ith long time slot. The index of all subtasks of the nth task is represented as $\mathcal{M}_{n,i} = \{1, 2, \ldots, M_{n,i}\}$. The task size of the mth subtask of the nth task at the ith long time slot is denoted by $D_{n,m,i}$.

Each vehicle sends a task-offloading request to the RSU, and the RSU designs a two-timescale joint resource allocation and subtask-offloading decisions for each vehicle, as shown in Fig. 2. Specifically, at each long time scale, bandwidth and computing resource allocation decisions are made; at each short time scale, subtask-offloading decisions are determined based on the acquired channel state information. The small-scale fading coefficient remains constant within one short time slot Δt, but fluctuates as an independent and identically distributed (i.i.d.) random variable over different short time slots. The vehicles offload subtasks to the VEC server for processing via vehicle-to-infrastructure (V2I) communication based on the offloading decision. For clarity of the following analysis, the key variables used in the system model are summarized in Nomenclature.

B. Vehicle's Mobility Model

In the considered vehicular network, the link connection time between the vehicle and the RSU plays a crucial role in guaranteeing successful offloading. The definition of link connection time is first given. After that, the model of link connection time is presented. As shown in Fig. 1, without loss of generality, we assume that the location coordinates of the nth vehicle to be $(x_{n,i}, y_{n,i}, 0)$ at the ith long time slot. The speed of the nth vehicle is v_n. The link connection time between the vehicle and the RSU is defined as the time interval in which the direct communication link between the vehicle

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:06:43 UTC from IEEE Xplore. Restrictions apply.

and the RSU is continuously available [32]. Hence, the link connection time between the nth vehicle and the RSU at the ith time slot can be expressed as

$$T_{n,i}^{link} = \frac{2\sqrt{(L_R)^2 - \left(x_{n,i}\right)^2} - y_{n,i}}{v_n} \tag{1}$$

where L_R is the communication diameter of the RSU.

C. Local Processing Model

The task is first unequally partitioned into multiple subtasks locally, and then some of these subtasks are processed locally. Since the task partitioning delay is small compared to the task-processing delay, it is ignored [33].

1) Task Partitioning: It is assumed that the nth task is unevenly partitioned into $M_{n,i}$ subtasks at the ith long time slot, which can be expressed as

$$\sum_{m=1}^{M_{n,i}} D_{n,m,i} = D_{n,i}. \tag{2}$$

If the total number of the subtasks offloaded by the vehicle is $M_{n,i}^*$, $(0 \le M_{n,i}^* \le M_{n,i})$, the number of the subtasks processed locally is $D_{n,i} - \sum_{m=1}^{M_{n,i}^*} D_{n,m,i}$.

2) Local Computing: The computing delay for processing subtasks locally at the nth vehicle in the ith long time slot can be expressed as

$$t_{n,i}^{lo} = \frac{\left(D_{n,i} - \sum_{m=1}^{M_{n,i}^*} D_{n,m,i}\right)\delta_{n,i}}{F_n^L} \tag{3}$$

where F_n^L is the CPU frequency of the nth vehicle.

D. Task-Offloading Model

The vehicle transmits some subtasks to the RSU and then forwarded to the VEC server for processing.

1) Uplink Transmission: The orthogonal frequency division multiple access (OFDMA) is adopted as the channel access scheme [34], [35]. We consider the channels affected by block-fading Rayleigh [36], [37].

Based on the lengths of the long and short time slots, the total number of short time slots can be calculated as $J = \Delta T/\Delta t$. Therefore, the uplink transmission rate at the jth short time slot between the nth vehicle and the RSU can be expressed as

$$R_{n,i,j} = b_{n,i}B_{total}\log_2\left(1 + \frac{P_n g_{n,i,j}}{\sigma^2}\right) \tag{4}$$

where B_{total} denotes the total bandwidth of the uplink channel, $b_{n,i}$ is the bandwidth allocation ratio at the ith long time slot, P_n represents the transmission power of the nth vehicle, σ^2 denotes the additive noise power, $g_{n,i,j} = |h_{n,i,j}|^2(d_{n,i,j})^{-\gamma}$ is the uplink channel gain between the nth vehicle and the RSU at the jth short time slot, and $h_{n,i,j}$ is the channel fading coefficient from the nth vehicle and the RSU at the jth short time slot. $d_{n,i,j}$ is the distance between the nth vehicle and the RSU at the jth short time slot, which is

$$d_{n,i,j} = \sqrt{\left(x_{n,i} + x_R\right)^2 + \left(y_{n,i} - L_R - \triangle y\right)^2 + H^2} \tag{5}$$

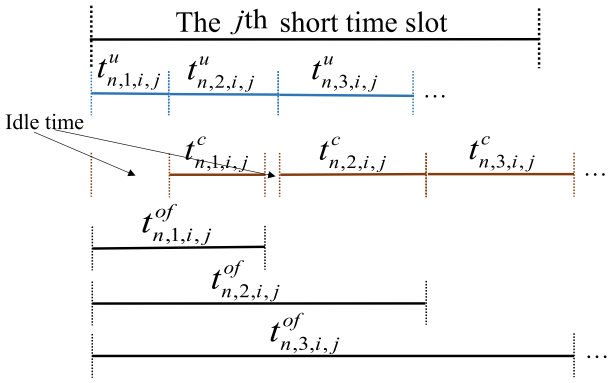


Fig. 3. Graphical analysis of the subtask uploading delay and computing delay in the jth short time slot.

where $\triangle y = v_n((i-1)\Delta T + (j-1)\Delta t$, x_R is the horizontal distance from the RSU to the road, L_R is the communication radius of the RSU, and H is the vertical height of the RSU.

It is assumed that the maximum number of subtasks that are uploaded to the VEC server at the jth short time slot can be denoted as

$$\sum_{m=1}^{M_{n,i,j}} D_{n,m,i} = R_{n,i,j}\Delta t \tag{6}$$

where $M_{n,i,j}$ is the maximum number of subtasks uploaded to the VEC server at jth short time slot.

The uploading delay of the mth subtask at the jth short time slot from the nth vehicle to the RSU is

$$t_{n,m,i,j}^u = \frac{D_{n,m,i}}{R_{n,i,j}}. \tag{7}$$

2) Computing at the VEC Server: For a subtask, the computing delay for the mth subtask at the VEC server is

$$t_{n,m,i,j}^c = \frac{D_{n,m,i}\delta_n}{f_{n,i}} \tag{8}$$

where $f_{n,i}$ is the CPU frequency allocated by the VEC server to the nth vehicle at the ith long time slot.

3) Offloading Delay at One Short Time Slot: Each subtask can only be processed by the VEC server after it has been successfully uploaded to the VEC server.

As shown in Fig. 3, the second subtask is processed when both conditions are met: 1) the second subtask has been successfully uploaded to the VEC server, and 2) the first subtask has been fully processed. At the jth short time slot, if the first subtask has already been processed, while the second subtask has not yet arrived at the VEC server, the VEC server must wait until the second subtask has fully arrived before it can be processed. While the VEC server is waiting, the computing resources allocated to the nth vehicle are idle, which is denoted as $t_{n,2,i,j}^{idle} = \sum_{r=1}^{2} t_{n,r,i,j}^u - t_{n,1,i,j}^{of}$, where $t_{n,1,i,j}^{of} = t_{n,1,i,j}^u + t_{n,1,i,j}^c$. Therefore, the processing delay for processing the second subtask is $t_{n,2,i,j}^{of} = \sum_{r=1}^{2} t_{n,r,i,j}^u + t_{n,2,i,j}^c$, $\sum_{r=1}^{2} t_{n,r,i,j}^u > t_{n,1,i,j}^{of}$. When processing the third subtask, it has already arrived at the VEC server. After the second subtask is finished, the VEC server proceeds directly to handle the third one. At this moment, the VEC server is not idle. The

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:06:43 UTC from IEEE Xplore. Restrictions apply.

processing delay for completing the third subtask is expressed as $t^{of}_{n,3,i,j} = t^{of}_{n,2,i,j} + t^{c}_{n,3,i,j}, \sum_{r=1}^{3} t^{u}_{n,r,i,j} \leqslant t^{of}_{n,2,i,j}$.

Therefore, when the number of subtasks offloaded at the jth short time slot is $\beta_{n,i,j}$, $0 \leqslant \beta_{n,i,j} \leqslant M_{n,i,j}$, then the task-processing delay for completing these subtasks in the jth short time slot is

$$t^{of}_{n,i,j} = \begin{cases} \sum_{m=1}^{\beta_{n,i,j}} t^{u}_{n,m,i,j} + t^{c}_{n,\beta_{n,i,j},i,j} \\ \sum_{m=1}^{\beta_{n,i,j}} t^{u}_{n,m,i,j} > t^{of}_{n,\beta_{n,i,j}-1,i,j} \\ t^{of}_{n,\beta_{n,i,j}-1,i,j} + t^{c}_{n,\beta_{n,i,j},i,j} \\ \sum_{m=1}^{\beta_{n,i,j}} t^{u}_{n,m,i,j} \leqslant t^{of}_{n,\beta_{n,i,j}-1,i,j}. \end{cases} \tag{9}$$

The idle time in the jth short time slot is

$$t^{idle}_{n,i,j} = \begin{cases} t^{idle}_{n,\beta_{n,i,j}-1,i,j} + \left(\sum_{m=1}^{\beta_{n,i,j}} t^{u}_{n,m,i,j} - t^{of}_{n,\beta_{n,i,j}-1,i,j} \right) \\ \sum_{m=1}^{\beta_{n,i,j}} t^{u}_{n,m,i,j} > t^{of}_{n,\beta_{n,i,j}-1,i,j} \\ t^{idle}_{n,\beta_{n,i,j}-1,i,j}, \sum_{m=1}^{\beta_{n,i,j}} t^{u}_{n,m,i,j} \leqslant t^{of}_{n,\beta_{n,i,j}-1,i,j}. \end{cases} \tag{10}$$

III. Problem Formulation

This study aims to minimize the average processing delay for all tasks by optimizing bandwidth and computational resources over long time slots and optimizing the number of offloaded subtasks over short time slots. The processing delay of subtasks at the VEC server across multiple short time slots is first analyzed within each long time slot. Then, the average processing delay of the all tasks minimization problem is formulated.

A. Processing Delay at the VEC Server

The processing order of subtasks follows a first-come, and first-served principle. Before processing subtasks uploaded within the current short time slot, all subtasks uploaded during the previous short time slot must be processed first.

If the current time slot is the jth short time slot, the previous short time slot is the $(j-1)$th short time slot. The remaining subtasks of the VEC server within the $(j-1)$th short time slot will continue to be processed within the jth short time slot. The computing delay for processing the remaining subtasks of the $(j-1)$th short time slot is

$$t^{Q}_{n,i,j-1} = max\left(t^{of}_{n,i,j-1} - \Delta t, 0\right). \tag{11}$$

$t^{Q}_{n,i,j-1}$ will occupy the idle time within the jth short time slot, which will affect the number of unprocessed subtasks in the current short time slot. According to $t^{Q}_{n,i,j-1}$ and $t^{idle}_{n,i,j}$, assuming that the subtasks are uploaded within the jth short time slot, the processing delay of these subtasks at the VEC server can be categorized into two cases.

1) *Case 1:* If $t^{Q}_{n,i,j-1} < t^{idle}_{n,i,j}$, the processing delay at the VEC server is

$$t^{of}_{n,i} = (j-1)\Delta t + t^{of}_{n,i,j}. \tag{12}$$

2) *Case 2:* If $t^{Q}_{n,i,j-1} \geqslant t^{idle}_{n,i,j}$, the processing delay can be expressed as

$$t^{of}_{n,i} = (j-1)\Delta t + t^{Q}_{n,i,j-1} + \frac{\delta_{n,i} \sum_{m=1}^{\beta_{n,i,j}} D_{n,m,i}}{f_{n,i}}. \tag{13}$$

B. Task-Processing Delay Minimization

Since each task is processed simultaneously by the vehicle and the VEC server at the ith long time slot, the task-processing delay is the longer of the local processing delay and the processing delay at the VEC server, which can be expressed as

$$t_{n,i} = \max\{t^{of}_{n,i}, t^{lo}_{n,i}\}. \tag{14}$$

Since our objective is to minimize the average processing delay of all tasks, the optimization problem can be expressed as

$$\begin{aligned} \mathcal{P}: \min_{\boldsymbol{b_i}, \boldsymbol{f_i}, \boldsymbol{\beta_i}} \ & \frac{1}{I} \sum_{i=1}^{I} \left(\frac{1}{N_i} \sum_{n=1}^{N_i} t_{n,i} \right) \\ \text{s.t.} \ & C1: 0 \leqslant b_{n,i} \leqslant 1 \\ & C2: \sum_{n=1}^{N_i} b_{n,i} \leqslant 1 \\ & C3: 0 \leqslant f_{n,i} \leqslant F^{max}_{vec} \\ & C4: \sum_{n=1}^{N_i} f_{n,i} \leqslant F^{max}_{vec} \\ & C5: 0 \leqslant \beta_{n,i,j} \leqslant M_{n,i,j} \\ & C6: 0 \leqslant \sum_{j=1}^{J} \beta_{n,i,j} \leqslant M_{n,i} \\ & C7: t_{n,i} \leqslant \min\{\tau^{max}_{n,i}, T^{link}_{n,i}\} \end{aligned} \tag{15}$$

where $\boldsymbol{b_i} = [b_{1,i}, \ldots, b_{n,i}, \ldots, b_{N_i,i}]$ is the bandwidth allocation ratio vector for all tasks at the ith long time slot, $\boldsymbol{f_i} = [f_{1,i}, \ldots, f_{n,i}, \ldots, f_{N_i,i}]$ denotes the computing resources allocation vector for all tasks at the ith long time slot, $\boldsymbol{\beta_i} = [\boldsymbol{\beta_{1,i}}, \ldots, \boldsymbol{\beta_{n,i}}, \ldots, \boldsymbol{\beta_{N_i,i}}]$ is the number vector of the uploaded subtasks for all tasks at the ith long time slot, $\boldsymbol{\beta}_{n,i} = [\beta_{n,i,1}, \ldots, \beta_{n,i,j}, \ldots, \beta_{n,i,J}]$ represents the number vector of the uploaded subtasks for the nth task at the jth short time slot, C1 and C2 denote the bandwidth allocation ratio constraint, C3 and C4 constraint the VEC server CPU frequency, C5and C6 are the number constraint for uploading subtasks, and C7 ensures that the task-processing delay of the nth task does not exceed its maximum tolerable delay and the link connection time between the nth vehicle and the RSU.

It is evident that task offloading and resource allocation are optimized at different timescales, where the number of offloaded subtasks is optimized over a short time slot, while resource allocation is optimized over a long time slot to achieve this objective. Moreover, factors such as the high mobility of vehicles, the time-varying nature of wireless

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:06:43 UTC from IEEE Xplore. Restrictions apply.

channel conditions, and the dynamic number of vehicles within a cell make it a challenge for the system to obtain an effective task-offloading strategy in real time. Therefore, we employ a DRL approach to address this issue.

IV. Problem Solution

In this section, we first reformulate problem P as an MDP. The definitions for state, action, and reward are provided. Then, a DDPG-based TTORA scheme is proposed. By optimizing the number of subtask-offloading decisions within short time slots while allocating bandwidth and computing resources over long time slots, the average processing delay of all tasks is minimized.

A. Problem Transformation Based on MDP

The dynamic environment and the optimization problem P can be transformed into an MDP $(\mathcal{S}; \mathcal{A}; \mathcal{R})$. We treat the VEC server as an agent, and the detailed definitions of state, action, and reward are as follows.

State: At the beginning of each long time slot, the agent collects information about the current state environment. The state of the VEC network s_i is composed of the state of all vehicles, that is,

$$\boldsymbol{s_i} = \left[s_{1,i}, \ldots, s_{n,i}, \ldots s_{N_i,i}\right] \tag{16}$$

where $s_{n,i}$ is the state of the nth vehicle at the ith long time slot. State $s_{n,i} \in \boldsymbol{s_i}$ at the ith long time slot can be defined as

$$s_{n,i} = \{\boldsymbol{task_{n,i}}, \boldsymbol{g_{n,i}}\} \tag{17}$$

where $\boldsymbol{task_{n,i}} = (D_{n,i}, \delta_{n,i}, \tau_{n,i}^{max})$ is the task attributes of the nth vehicle at the ith long time slot, $\boldsymbol{g_{n,i}} = [g_{n,i,1}, \ldots, g_{n,i,j}, \ldots, g_{n,i,J}]$ represents the channel gain vector between the nth vehicle and the RSU.

At the beginning of each short time slot, the agent acquires the channel gain between the vehicles and the RSU. $g_{n,i,j}$ is the channel gain between the nth vehicle and the RSU at the jth short time slot. $g_{n,i,j}$ is the channel gain between the nth vehicle and the RSU at the jth short time slot.

Action: Based on $\boldsymbol{s_i}$, the agent determines the action for all vehicles at the ith long time slot

$$\boldsymbol{a_i} = \left[a_{1,i}, \ldots, a_{n,i}, \ldots a_{N_i,i}\right] \tag{18}$$

where $a_{n,i}$ is the action of the nth vehicle, which includes bandwidth allocation ratio $b_{n,i}$, computing resource allocation $f_{n,i}$ and total number of subtasks offloaded $M_{n,i}^*$. The action is defined as

$$a_{n,i} = \{b_{n,i}, f_{n,i}, M_{n,i}^*\}. \tag{19}$$

At the beginning of each short time slot, based on the channel gain between the vehicles and the RSU, the agent optimizes the number of subtasks offloaded, which is $\boldsymbol{\beta_{n,i}} = [\beta_{n,i,1}, \ldots, \beta_{n,i,j}, \ldots, \beta_{n,i,J}]$, where $\boldsymbol{\beta_{n,i}}$ is the vector of number of subtasks offloaded, and $\beta_{n,i,j}$ is the number of subtasks offloaded at the jth short time slot.

Based on the channel gain, the agent can determine the maximum number of offloading subtasks for each short time slot. If the subtask offloading is completed within the jth short time slot, the number of subtasks offloaded at each short time slot can be expressed as

$$\beta_{n,i,r} = \begin{cases} M_{n,i,r}, & r < j \\ M_{n,i}^* - \sum_{r=1}^{j-1} M_{n,i,r}, & r = j \\ 0, & r > j. \end{cases} \tag{20}$$

Reward: To minimize the average processing delay of all tasks, the reward function r_i is defined in two parts: 1) the negative of the average processing delay of all tasks if all constraints are satisfied, and 2) a negative penalty for failing to satisfy the delay constraint. This reward can be expressed as

$$r_i = \begin{cases} -\frac{1}{N_i} \sum_{n=1}^{N_i} t_{n,i}, & all\ constraints\ satisfied \\ -\eta, & Otherwise \end{cases} \tag{21}$$

where η denotes the penalty related to the constraints, which is a constant.

B. DDPG-Based TTORA Algorithm

Given our model, we adopt a DDPG-based algorithm to solve the formulated optimization problem. According to the actor–critic architecture, the DDPG-based algorithm is decomposed into four components: the current actor network, the target actor network, the current critic network, and the target critic network. The parameters of networks are defined: θ^{μ} and $\theta^{\mu'}$ are the parameters of the actor network and actor target network, respectively; θ^{Q} and $\theta^{Q'}$ are the parameters of the critic network and critic target network, respectively.

To obtain the resource allocation decision $(b_{n,i}, f_{n,i})$ at each long time slot and subtask-offloading decision $(\boldsymbol{\beta_{n,i}})$ at each short time slot, a DDPG-based TTORA algorithm is proposed to tackle the MDP, as summarized in Algorithm 1. The actor network θ^{Q} and critic network θ^{μ} are initialized. The agent collects information from the current environment to form a state space $\boldsymbol{s_i}$. The actor network takes action $\boldsymbol{a_i}$ based on the current $\boldsymbol{s_i}$. After executing the selected action $\boldsymbol{a_i}$, the agent obtains the next state $\boldsymbol{s_{i+1}}$ and the reward r_i. The tuple $(\boldsymbol{s_i}, \boldsymbol{a_i}, r_i, \boldsymbol{s_{i+1}})$ is stored in the experience replay buffer. K transitions form a mini-batch and input them into the actor network and critic network for training. According to the current state and action $(\boldsymbol{s_i}, \boldsymbol{a_i})$, $Q(\boldsymbol{s_i}, \boldsymbol{a_i}|\theta^{Q})$ can be calculated by the critic network to minimize the loss function. The actor network updates the current policy using the policy gradient. Then, the parameters of the target network are updated.

From the above, the action at the ith long time slot can be expressed as

$$\boldsymbol{a_i} = \mu\left(\boldsymbol{s_i} \mid \theta^{\mu}\right) + \xi_i \tag{22}$$

where ξ_i is random noise to enable the agent to explore potentially better strategies. K transitions tuple $(\boldsymbol{s_i}, \boldsymbol{a_i}, r_i, \boldsymbol{s_i} + \boldsymbol{1})$ sampled uniformly from the experience-replay buffer and transferred them to the actor and critic networks. The target Q value is given by

$$y_i = r_i + \phi Q'\left(\boldsymbol{s_{i+1}}, \mu'\left(\boldsymbol{s_{i+1}}|\theta^{\mu'}\right)|\theta^{Q'}\right) \tag{23}$$

where ϕ is the discount factor of the future reward.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:06:43 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 DDPG-Based Two-Timescale Task Offloading and Resource Allocation (DDPG-TTORA)

Input: State space $\mathcal{S}$, action space $\mathcal{A}$ and the parameters about the testbed environments;

Output: Bandwidth and computing resource allocation at each long time slot, number of subtasks offloaded at each short time slot;

1 Initialize the actor and critic network parameters: θ^{Q}, θ^{μ};
2 Initialize the target actor and critic network parameters: $\theta^{Q'}$, $\theta^{\mu'}$;
3 Initialize the experience replay buffer
4 **for** each episode $e = 1, 2, \ldots, E$ **do**
5 Reset initial environment state $\boldsymbol{s_0}$;
6 **for** $i = 1, 2, \ldots, I$ **do**
7 Choose an action generated by the actor network θ^{μ} and additional noise ξ_i: $\boldsymbol{a_i} = \mu(s_i \mid \theta^{\mu}) + \xi_i$;
8 Perform action $\boldsymbol{a_i}$, get rewards r_i and the next state $\boldsymbol{s_{i+1}}$;
9 Store the transitions tuples $(\boldsymbol{s_i}, \boldsymbol{a_i}, r_i, \boldsymbol{s_{i+1}})$ in the replay buffer;
10 **if** number of tuples in the replay buffer is larger than K **then**
11 Randomly sample a mini-batch of K transitions tuples $(\boldsymbol{s_i}, \boldsymbol{a_i}, r_i, \boldsymbol{s_{i+1}})$ from the replay buffer and transfer it to the actor and critic networks;
12 Set $y_i = r_i + \phi Q'(\boldsymbol{s_{i+1}}, \mu'(\boldsymbol{s_{i+1}}|\theta^{\mu'})|\theta^{Q'})$;
13 Update the actor network using the gradient of the critic network based on Eq. (24);
14 Update the critic network by minimizing the loss function based on Eq. (25);
15 Soft update the parameters of the target network based on Eq. (26);
16 **end if**
17 **end for**
18 **end for**

The action value of $(\boldsymbol{s_i}, \boldsymbol{a_i})$ is evaluated by the critic network and denoted as $Q(\boldsymbol{s_i}, \boldsymbol{a_i}|\theta^{Q})$. The actor network is updated using a gradient method, which can be expressed as

$$\theta^{\mu} \leftarrow \theta^{\mu} - LR_{\mu} \bigtriangledown_{\theta} \mathcal{J}\left(\theta_{\mu}\right) \tag{24}$$

where LR_{μ} is the learning rate of the actor network, $\bigtriangledown_{\theta}\mathcal{J}(\theta_{\mu}) \approx 1/K \sum_{k=1}^{K} \bigtriangledown_{a} Q(s, a|\theta^{Q})|_{s=s_i, a=\mu(s_i)} \bigtriangledown_{\theta^{\mu}} \mu(s|\theta^{\mu})|_{s_i}$ is the gradient of $Q(\boldsymbol{s_i}, \boldsymbol{a_i}|\theta^{Q})$.

Then, the parameters of the critic network can be updated by minimizing the mean square error, which can be defined as

$$\theta^{Q} \leftarrow \theta^{Q} - LR_{Q} \bigtriangledown_{\theta} \left[\frac{1}{K}\sum_{k=1}^{K} y_i - Q\left(\boldsymbol{s_i}\boldsymbol{a_i}|\theta^{Q}\right)\right]^2 \tag{25}$$

where LR_Q is the learning rate of the critic network.

Finally, the update process of the target network can be expressed as

$$\begin{aligned} \theta^{\mu'} &\leftarrow \tau\theta^{\mu} + (1-\tau)\,\theta^{\mu'} \\ \theta^{Q'} &\leftarrow \tau\theta^{Q} + (1-\tau)\,\theta^{Q'} \end{aligned} \tag{26}$$

where $\tau \in (0, 1)$ is the soft-update rate.

Remark 1: (Complexity of Algorithm 1): The computational complexity of Algorithm 1 mainly depends on parameter updating for four networks (actor network, critic network, target actor network, and target critic network) and gradient computations of two networks (actor network and critic network). The computational complexity of calculating the gradient for the actor network and the critic network are defined by G_A and G_C, respectively, and the computational complexity of updating the parameters for the actor network and the critic network are defined by U_A and U_C, respectively. During each iteration, Algorithm 1 is trained only when the tuples stored in the reply buffer exceed K. Therefore, the number of training for Algorithm 1 is EI–K, where E is the algorithm training episode, each episode contains I long time slots for training, and K represents the number of transition tuples in each mini-batch. In each training step, the computational complexity of steps 11–15 is $\mathcal{O}(G_A + G_C + 2U_A + 2U_C)$. Thus, the computational complexity of Algorithm 1 can be given as $\mathcal{O}((E'I - K)(G_A + G_C + 2U_A + 2U_C))$.

Remark 2: (Convergence of Algorithm 1): Since the reward defined in (21) is bounded by the delay constraint C7 and the constant penalty term, the Bellman backup in (23) is well defined on the space of bounded action-value functions. For any fixed target policy μ', the Bellman operator associated with (23) is a ϕ-contraction with $0 < \phi < 1$, and therefore admits a unique bounded fixed point $Q^{\mu'}$. Under standard stochastic approximation assumptions, including bounded gradients, sufficiently small learning rates, and sufficiently rich replay samples, the critic update in (25) converges to a stationary solution of the Bellman residual minimization, while the actor update in (24) converges to a stationary point of the expected return. Moreover, the soft target-network update in (26) ensures smooth tracking of the online networks and improves training stability. Hence, Algorithm 1 converges to a locally stationary solution, i.e., a locally stable point.

V. Simulation Results and Analysis

To demonstrate the superiority of the proposed DDPG-TTORA algorithm, extensive simulations are conducted. First, the convergence of the proposed DDPG-TTORA algorithm is analyzed. Subsequently, its impact on system performance is evaluated under various conditions, such as different task scales, different task computational densities, different numbers of vehicles, and different computing capabilities of the VEC server.

A. System Settings

In doing the simulations, a multivehicle VEC network consisting of one RSU, a VEC server, and multiple vehicles is considered. The number of vehicles is set within the range of [4, 14]. The number of vehicles arriving at the cell follows a Poisson process, with mean rates of $\lambda = 0.1$ s^{-1}. These vehicles are randomly distributed on a unidirectional two-lane road covered by the RSU. Each vehicle has a task, where the task size, task computation density, and maximum tolerable delay are randomly generated and scaled into the intervals [0.5, 3] Mbits, [200, 1000] cycles/bit, and [200, 800] ms,

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:06:43 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PARAMETER SETTING

Parameter	Value
The RSU's height to ground H	20 m
The horizontal distance of RSU x_R	3 m
The RSU's coverage radius L_R	200 m
The vehicle speed v_n	[30,70] Km/h
The CPU frequency of the VEC server F_{vec}^{max}	[20,70] GHz
The CPU frequency of each vehicle F_n^L	1 GHz
The transmission power of the vehicles P_n	23 dBm
The total bandwidths of the uplink channels B_{total}	10 MHz
The path loss exponent γ	3
The white Gaussian noise powers σ^2	-70 dBm
The long time slot ΔT	1 s
The short time slot Δt	10 ms

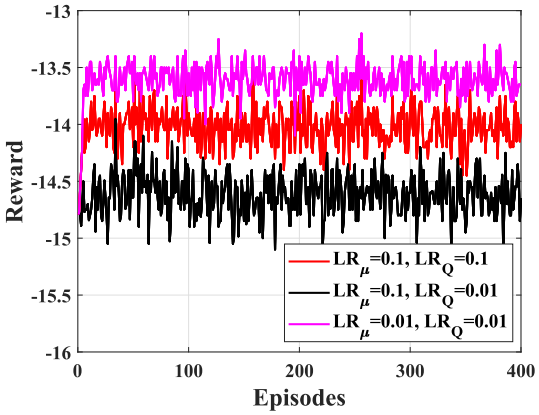


Fig. 4. Convergence of the proposed algorithm under different learning rates.

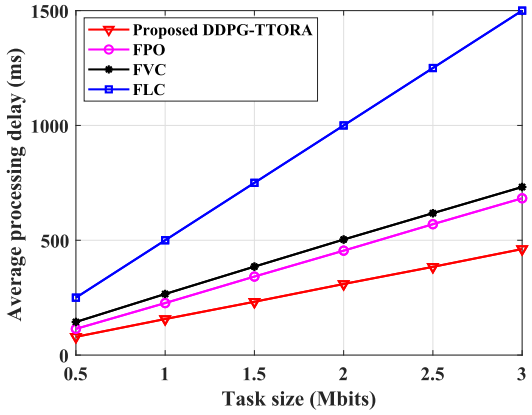


Fig. 5. Average processing delays of the proposed DDPG-TTORA, FPO, FVC, and FLC for different task sizes.

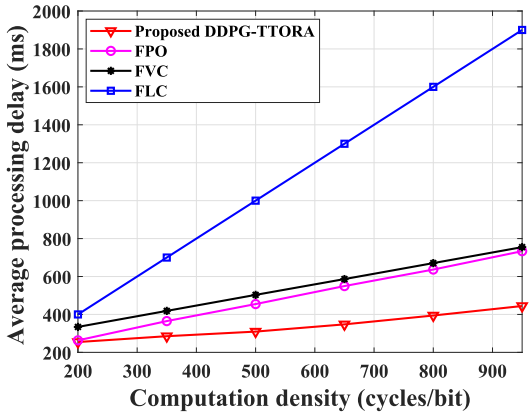


Fig. 6. Average processing delays of the proposed DDPG-TTORA, FPO, FVC, and FLC for different task computation densities.

respectively. Each task is divided into multiple unequal subtasks. The size range of each subtask is set to [15, 35] kb [26]. During training, the experience replay buffer size, mini-batch size, and the discount factor are set to 10 000, 64, and 0.99. Other key parameter settings are detailed in Table I.

B. Convergence of the Proposed DDPG-TTORA

The convergence of the proposed DDPG-TTORA is investigated under different learning rates, i. e., $LR_\mu = \{0.1, 0.01\}$, $LR_Q = \{0.1, 0.01\}$, when the task size is all set to be 2 Mbits, the task computation density is set to be 500 cycles/bit for all tasks, the number of the vehicles is set to be 10 and the computing capability of the VEC server is set to be 50 GHz. DDPG-TTORA exhibits good convergence performance as shown in Fig. 4. It is also shown that when $LR_\mu = \{0.01\}$ and $LR_Q = \{0.01\}$, DDPG-TTORA performs best. Therefore, we use these values for the rest of the simulations.

C. Delay Performance of the Proposed DDPG-TTORA

To demonstrate the superiority of the proposed DDPG-TTORA, we compare it with four existing schemes in the VEC networks.

1) *FPO:* Each vehicle offloads a portion of its task to the VEC server for processing, while the remaining part is processed locally. The task is handled in parallel both locally and at the VEC server. By optimizing the number of subtasks offloaded, bandwidth, and computational resource allocation decisions at each long time slot minimize, the average processing delay for all tasks [25], [38].
2) *Full VEC Computing (FVC):* All tasks are uploaded to the VEC server for processing, minimizing the average processing delay of all tasks by optimizing bandwidth and computational resource allocation decisions at each long time slot.
3) *Full Local Computing (FLC):* All tasks are processed locally.

Different from DDPG-TTORA, both FPO and FVC schemes adopt a serial processing method for subtask uploading and computation, without considering parallel processing.

The simulation results are shown in Figs. 5–8.

1) Impact of the Task Size: The average processing delays of all the algorithms for different task sizes are shown in Fig. 5. It is seen that for the given communication and computing resources of the RSU, as the task size increases, the task uploading delay and the computing delay increase, thereby

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:06:43 UTC from IEEE Xplore. Restrictions apply.

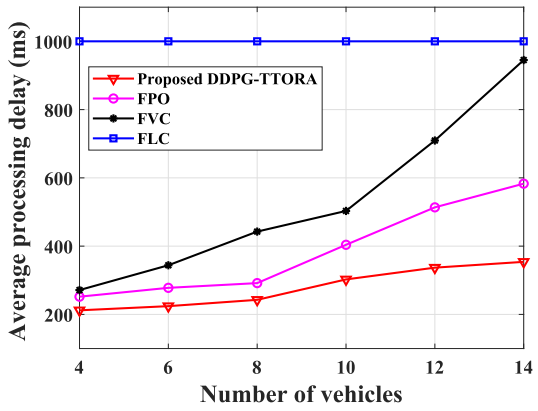


Fig. 7. Average processing delays of the proposed DDPG-TTORA, FPO, FVC, and FLC for different numbers of vehicles.

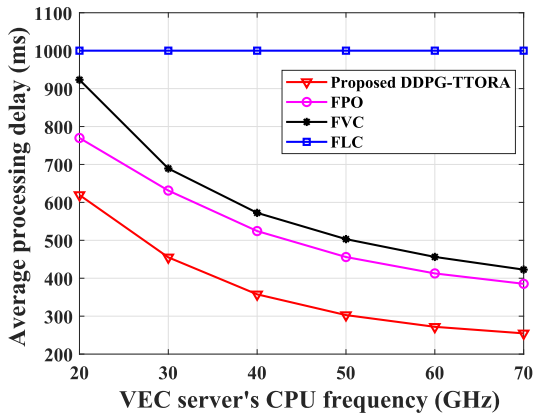


Fig. 8. Average processing delays of the proposed DDPG-TTORA, FPO, FVC, and FLC for different computing capabilities of the VEC server.

increasing the task-processing delay for each vehicle. It is worth noting that the system delay with the proposed DDPG-TTORA is always the minimum compared to those of the simulated algorithms. This is because the proposed DDPG-TTORA utilizes local and VEC server's computing resources, while FLC and FVC utilize local computing resources and VEC server's computing resources, respectively. Compared with the FPO, the proposed DDPG-TTORA significantly reduces idle time for the VEC server's computing resources, thereby improving the utilization rate of the VEC server's computing resources. This is because the FPO uploads all subtasks before performing computation, leaving the VEC server's computing resources idle during the uploading process, which results in low resource utilization. Taking the task size of 2 Mbits as an example, the average processing delay with DDPG-TTORA is about 31.93% less than that with FPO. This is because the proposed DDPG-TTORA employs a parallel mechanism for subtask uploading and computing, which improves the utilization rate of the VEC server's computing resources.

2) Impact of the Task Computation Density: The average processing delays of those performances are also investigated for different task computation densities, and the results are shown in Fig. 6. It can be observed that under fixed communication and computing resources of the RSU, an increase in task computation density leads to higher task computing delay, thereby resulting in increased processing delay for each task. The results also show that the average processing delay with the proposed DDPG-TTORA is always the minimum. Taking the task computation density of 650 cycles/bit as an example, the average processing delay with the proposed DDPG-TTORA is about 36.75% less than that of FPO. This is because DDPG-TTORA can make full use of the computing resources of the VEC server.

In addition, jointly observing Figs. 5 and 6, we see that as the task size/task computation density increases, the average processing delay gap between DDPG-TTORA and other baselines becomes much more obvious. For example, in Fig. 6, when the task computation density is 200 and 950 cycles/bit, compared to FPO, DDPG-TTORA reduces the average processing delay by about 3.37% and 39.42%, respectively. This is because, for the given computing resources of the VEC server, as the task computation density increases, the computing delay for processing subtasks also increases. When the task-processing delay is primarily dominated by task computing delay, improving the computing resource utilization of the VEC server can significantly reduce task-processing delay.

3) Impact of the Number of the Vehicles: Fig. 7 shows the average processing delays with the proposed DDPG-TTORA, FPO, and FVC increase as the number of vehicles increases. This is because, for the given communication and computing resources of the RSU, as the number of vehicles increases, the bandwidth and computing resources allocated to each vehicle by the RSU decrease, leading to an increase in the average processing delay. The average processing delay with FLC remains constant. This is because, for the given computing resources of each vehicle, increasing the number of vehicles does not affect the computing resources allocated to each vehicle. With fixed task size and task computation density, since the tasks of all vehicles are processed locally, the average processing delay of FLC remains unchanged. It is worth noting that the average processing delay with the proposed DDPG-TTORA is always the minimum. The reason is similar to that for Fig. 5.

4) Impact of the Computing Capability of the VEC Server: The average processing delays with the proposed DDPG-TTORA, FPO, and FVC first decrease and then slowly decrease as the computing capability of the VEC server increases, as shown in Fig. 8. This is because, with a fixed number of vehicles, the more computing resources allocated to the VEC server, the greater the computing resources available to each vehicle, thereby reducing average processing delay. When the computing resources of the VEC server further increase, due to the given communication resources, the subtask computing delay is much smaller than the subtask transmission delay. The task-processing delay is mainly limited by the subtask transmission delay, so the reduction in task-processing delay is not very significant. In contrast, the average processing delay with FLC remains constant. This is because, for a given computing resource of each vehicle, increasing the computing resources of the VEC server does not affect the computing resources of each vehicle. Consequently, the local processing latency for all vehicles

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:06:43 UTC from IEEE Xplore. Restrictions apply.

remains unchanged. The results also show that the average processing delay with the proposed DDPG-TTORA is always the minimum. The reason is similar to that for Fig. 7.

VI. Conclusion

In this work, we proposed a novel partial offloading scheme by assuming that the vehicular task can be partitioned into unequal-sized subtasks. In contrast to existing work, we have taken nonarbitrary task partitioning into account and have studied the subtask uploading and computing parallel processing. Since the task generation time and small-scale channel gain variations occur at different time scales, a two-timescale task-offloading problem was formulated by jointly optimizing the number of subtask offloadings, communication, and computing resource allocation. The problem was transformed into an MDP. A DDPG-TTORA algorithm was proposed to obtain resource allocation decisions at long time slots and subtask-offloading decisions at short time slots. Simulation results demonstrated that the proposed DDPG-TTORA can effectively reduce the average processing delay compared to the other baseline algorithms.

References

[1] W. Xiao, X. Ling, M. Chen, J. Liang, S. A. AlQahtani, and M. Chen, "MVPOA: A learning-based vehicle proposal offloading for cloud-edge–vehicle networks," *IEEE Internet Things J.*, vol. 12, no. 5, pp. 4738–4749, Mar. 2025.

[2] A. M. Huroon, G. B. Tarekegn, A. M. A. Abdo, A. Amjad, L.-C. Tai, and L.-C. Wang, "Multiagent deep reinforcement learning for AAV-RIS-assisted integrated sensing and communication," *IEEE Internet Things J.*, vol. 12, no. 19, pp. 40083–40097, Oct. 2025.

[3] A. M. Huroon, Y.-C. Huang, and L.-C. Wang, "Energy-efficient transmission strategy for UAV-RIS 2.0 assisted communications using rate splitting multiple access," *IEEE Trans. Wireless Commun.*, vol. 25, pp. 5246–5261, 2026.

[4] X. Song, W. Zhang, L. Lei, X. Zhang, and L. Zhang, "UAV-assisted heterogeneous multi-server computation offloading with enhanced deep reinforcement learning in vehicular networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 6, pp. 5323–5335, Nov. 2024.

[5] H. Guo, X. Chen, X. Zhou, and J. Liu, "Trusted and efficient task offloading in vehicular edge computing networks," *IEEE Trans. Cognit. Commun. Netw.*, vol. 10, no. 6, pp. 2370–2382, Dec. 2024.

[6] L. Bai, J. Cao, M. Zhang, and B. Li, "Collaborative edge intelligence for autonomous vehicles: Opportunities and challenges," *IEEE Netw.*, vol. 39, no. 2, pp. 52–60, Mar. 2025.

[7] Y. Chen, J. Wu, J. Han, H. Zhao, and S. Deng, "A game-theoretic approach-based task offloading and resource pricing method for idle vehicle devices assisted VEC," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21954–21969, Jun. 2024.

[8] Y. Xia, H. Zhang, X. Zhou, and D. Yuan, "Location-aware and delay-minimizing task offloading in vehicular edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 12, pp. 16266–16279, Dec. 2023.

[9] Y. Wang, Z. Qian, Z. Yu, and F. Li, "Intelligent online computation offloading for wireless powered vehicle edge computing," in *Proc. ICC - IEEE Int. Conf. Commun.*, May 2023, pp. 925–930.

[10] F. Zhu, X. Liu, K. Yu, Q. Zhang, Z. Feng, and D. Li, "Delay-effective task offloading technology in Internet of Vehicles: From the perspective of the vehicle platooning," *IEEE Trans. Commun.*, vol. 73, no. 6, pp. 3833–3848, Jun. 2025.

[11] N. Fofana, A. B. Letaifa, and A. Rachedi, "Intelligent task offloading in vehicular networks: A deep reinforcement learning perspective," *IEEE Trans. Veh. Technol.*, vol. 74, no. 1, pp. 201–216, Jan. 2025.

[12] W. Fan, Y. Zhang, G. Zhou, and Y. Liu, "Deep reinforcement learning-based task offloading for vehicular edge computing with flexible RSU-RSU cooperation," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 7, pp. 7712–7725, Jul. 2024.

[13] Y. Zhang, C. Chen, H. Zhu, Y. Pan, and J. Wang, "Latency minimization for MEC-V2X assisted autonomous vehicles task offloading," *IEEE Trans. Veh. Technol.*, vol. 74, no. 3, pp. 4917–4932, Mar. 2025.

[14] X. He, S. Pang, H. Gui, K. Zhang, N. Wang, and S. Yu, "Online offloading and mobility awareness of DAG tasks for vehicle edge computing," *IEEE Trans. Netw. Service Manage.*, vol. 22, no. 1, pp. 675–690, Feb. 2025.

[15] D. Hu, M. Yuan, G. Huang, S. Zhao, and D. Tang, "Optimization of vehicular edge computing under time-varying fading channels with path prediction," *IEEE Internet Things J.*, vol. 12, no. 5, pp. 5500–5514, Mar. 2025.

[16] S. Wang, B. Yang, Z. Yu, X. Cao, Y. Zhang, and C. Yuen, "Computation offloading for multi-server multi-access edge vehicular networks: A DDQN-based method," in *Proc. IEEE 99th Veh. Technol. Conf. (VTC-Spring)*, Jun. 2024, pp. 1–5.

[17] W. Zhao, Y. Cheng, Z. Liu, X. Wu, and N. Kato, "Asynchronous DRL-based multi-hop task offloading in RSU-assisted IoV networks," *IEEE Trans. Cognit. Commun. Netw.*, vol. 11, no. 1, pp. 546–555, Feb. 2025.

[18] Q. Luo, J. Zhang, S. Hu, T. H. Luan, and P. Fan, "Joint task migration and resource allocation in vehicular edge computing: A deep reinforcement learning-based approach," *IEEE Trans. Veh. Technol.*, vol. 74, no. 6, pp. 9476–9490, Jun. 2025.

[19] Y. Li, L. Li, and P. Fan, "Mobility-aware computation offloading and resource allocation for NOMA MEC in vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 8, pp. 11934–11948, Aug. 2024.

[20] S. Tian, X. Zhu, B. Feng, Z. Zheng, H. Liu, and Z. Li, "Partial offloading strategy based on deep reinforcement learning in the Internet of Vehicles," *IEEE Trans. Mobile Comput.*, vol. 24, no. 7, pp. 6517–6531, Jul. 2025.

[21] G. Ma et al., "Joint partial offloading and resource allocation for vehicular federated learning tasks," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 8, pp. 8444–8459, Aug. 2024.

[22] Z. Abbas, S. Xu, and X. Zhang, "An efficient partial task offloading and resource allocation scheme for vehicular edge computing in a dynamic environment," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 2, pp. 2488–2502, Feb. 2025.

[23] Z. Lin, X. Chen, X. He, D. Tian, Q. Zhang, and P. Chen, "Energy-efficient cooperative task offloading in NOMA-enabled vehicular fog computing," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 7, pp. 7223–7236, Jul. 2024.

[24] D. Meng et al., "Incentive-driven partial offloading and resource allocation in vehicular edge computing networks," *IEEE Internet Things J.*, vol. 12, no. 8, pp. 11023–11035, Apr. 2025.

[25] S. Wu et al., "DRL-based pricing-driven for task offloading and dynamic resource in vehicle edge computing," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 10389–10404, Oct. 2025.

[26] X. Chen and G. Liu, "Energy-efficient task offloading and resource allocation via deep reinforcement learning for augmented reality in mobile edge networks," *IEEE Internet Things J.*, vol. 8, no. 13, pp. 10843–10856, Jul. 2021.

[27] B. Xie et al., "MOB-FL: Mobility-aware federated learning for intelligent connected vehicles," in *Proc. ICC - IEEE Int. Conf. Commun.*, May 2023, pp. 3951–3957.

[28] J. Huang, M. Zhang, J. Wan, Y. Chen, and N. Zhang, "Joint data caching and computation offloading in UAV-assisted Internet of Vehicles via federated deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 73, no. 11, pp. 17644–17656, Nov. 2024.

[29] H. Han et al., "Better late than never: GAN-enhanced dynamic anti-jamming spectrum access with incomplete sensing information," *IEEE Wireless Commun. Lett.*, vol. 10, no. 8, pp. 1800–1804, Aug. 2021.

[30] A. M. Huroon, Y.-C. Huang, and L.-C. Wang, "UAV-RIS assisted multiuser communications through transmission strategy optimization: GBD application," *IEEE Trans. Veh. Technol.*, vol. 73, no. 6, pp. 8584–8597, Jun. 2024.

[31] A. Gaur, A. Balakrishnan, S. Srirangarajan, S. De, P.-H. Tseng, and K.-T. Feng, "Optimal time partitioning in V2V integrated sensing and communication systems," *IEEE Wireless Commun. Lett.*, vol. 13, no. 12, pp. 3390–3394, Dec. 2024.

[32] W. Huang, Z. Zhao, G. Min, Y. Wang, and Z. Chang, "MMTO: Multi-vehicle multi-hop task offloading in MEC-enabled vehicular networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 11125–11136, Oct. 2025.

[33] J. Ren, G. Yu, Y. He, and G. Y. Li, "Collaborative cloud and edge computing for latency minimization," *IEEE Trans. Veh. Technol.*, vol. 68, no. 5, pp. 5031–5044, May 2019.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:06:43 UTC from IEEE Xplore. Restrictions apply.

[34] L. Zhao et al., "Novel Lagrange multipliers-driven adaptive offloading for vehicular edge computing," *IEEE Trans. Comput.*, vol. 73, no. 12, pp. 2868–2881, Dec. 2024.
[35] H. Li, X. Li, M. Zhang, and B. Ulziinyam, "System-wide energy efficient computation offloading in vehicular edge computing with speed adjustment," *IEEE Trans. Green Commun. Netw.*, vol. 8, no. 2, pp. 701–715, Jun. 2024.
[36] H. Maleki, M. Bas¸aran, and L. Durak-Ata, "Handover-enabled dynamic computation offloading for vehicular edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 7, pp. 9394–9405, Jul. 2023.
[37] Y. Xia, J. Tian, H. Zhang, and D. Yuan, "Joint task offloading and pricing strategy for multi-tier vehicular edge computing networks: A multi-leader multi-follower Stackelberg game approach," *IEEE Trans. Cognit. Commun. Netw.*, vol. 12, pp. 1877–1891, 2026.
[38] H. Tang, M. Du, H. Wu, P. Jiao, and R. Li, "TLCO: Topological link-aware task co-offloading method for joint V2V and V2I system," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 9, pp. 13247–13259, Sep. 2025.



Yang Xia (Member, IEEE) received the B.E. degree from the School of Electrical and Information Engineering, Anhui University of Science and Technology, Huainan, China, in 2015, the M.E. degree from the School of Electrical and Automation Engineering, Hefei University of Technology, Hefei, China, in 2018, and the Eng.D. degree from the School of Control Science and Engineering, Shandong University, Jinan, China, in 2025.

He is currently a Laboratory Technician with the School of Electrical and Automation Engineering, Hefei University of Technology. His research interests include vehicular networks, resource management, and game theory.



Lijuan Li (Graduate Student Member, IEEE) received the M.E. degree in electronic information from Qilu University of Technology, Jinan, Shandong, China, in 2024. She is currently pursuing the Eng.D. degree in energy and power engineering with Hefei University of Technology, Hefei, China.

Her research interests include active distribution networks, edge computing, and resource optimization.



Tong Xue (Member, IEEE) received the B.E. degree from the Electronic Engineering College, Heilongjiang University, Harbin, China, in 2017, the M.E. degree from the School of Electronics and Information Engineering, Harbin Institute of Technology, Harbin, in 2019, and the Ph.D. degree from the School of Control Science and Engineering, Shandong University, Jinan, China, in 2024.

She is currently a Lecturer with the College of Electronic and Information Engineering, Shandong University of Science and Technology, Qingdao, China. Her research interests include radio resource management and vehicle-to-everything (V2X) communications.



Daojun Liang received the B.S. degree in computer science from Taishan University, Tai'an, China, in 2016, and the M.S. and Ph.D. degrees from the School of Information Science and Engineering, Shandong Normal University, Jinan, China, in 2019 and 2025, respectively.

He is currently a Distinguished Associate Professor with Qilu University of Technology (Shandong Academy of Sciences), Jinan. His research interests include deep learning, machine learning, computer vision, and natural language processing.



Qiyue Li (Senior Member, IEEE) received the B.E. degree in electronic engineering from Wuhan University, Wuhan, China, in 2003, and the Ph.D. degree in communication and information system from the University of Science and Technology of China, Hefei, Anhui, China, in 2008.

From 2008 to 2011, he did a Post-Doctoral Research with the School of Computer Science and Technology, University of Science and Technology of China. He is currently a Professor with Hefei University of Technology, Hefei. His research interests include wireless sensor networks and indoor localization using wireless networks.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:06:43 UTC from IEEE Xplore. Restrictions apply.